

```
!mkdir -p ~/.kaggle
```

```
import json

kaggle_json = {
    "username": "--",
    "key": "--"
}

with open("/root/.kaggle/kaggle.json", "w") as f:
    json.dump(kaggle_json, f)
```

```
!chmod 600 ~/.kaggle/kaggle.json
```

```
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (2.0.0)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: kagglesdk<1.0,>=0.1.15 in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.1.16)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaggle) (26.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->kaggle) (1.1)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.11)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (2024.7.4)
```

```
!kaggle datasets list
```

ref	title	
nalisha/job-salary-prediction-dataset	Job Salary Prediction Dataset	31
ssssws/chocolate-sales-dataset-2023-2024	Chocolate Sales Dataset 2023 - 2024	244
grandmaster07/student-exam-performance-dataset-analysis	Student Exam Performance Dataset Analysis	
dhrub2028/student-academic-performance-dataset-for-ml	Student Academic Performance Dataset for ML	
rhythmghai/food-ordering-behavior-india-50k-orders	Food Ordering Behavior (India, 50K Orders)	10
obaidthere/amazon-bestsellers-price-vs-user-rating-analysis	Amazon Bestsellers: Price vs User Rating Analysis	
lucalullo/global-employment-unemployment-rates-1991-2025	global-employment-unemployment-rates-1991-2025	7
bhushandivekar/video-game-sales-and-industry-data-1980-2024	🎮 Video Game Sales & Industry Data (1980 - 2024)	:
minahilfatima12328/global-fuel-price-trends-1970-2026	Global Fuel Price Trends (1970-2026)	
sehaj1104/student-mental-health-and-burnout-dataset	Student Mental Health and Burnout Dataset	33
kandeelai22/messy-e-commerce-sales-dataset	Messy E-Commerce Sales Dataset	
algozee/global123445	Global Job Markets: Trends & Projections	7
rhythmghai/netflix-user-watching-behavior-dataset	📺 Netflix User Watching Behavior Dataset	:
nitikachandel95/student-social-media-impact-dataset	Student Social Media & Brain Rot Dataset	1422
kanchana1990/kentucky-real-estate-data-2026	Kentucky Real Estate Data 2026	24
emanfatima2/spotify-global-hits-and-artist-analytics	Spotify Global Hits and Artist Analytics	
alitaqishah/spotify-wrapped-2025-top-songs-and-artists	Spotify Wrapped 2025 Top Songs & Artists	
jayjoshi37/sleep-screen-time-and-stress-analysis	Sleep, Screen Time and Stress Analysis	3
kaushalnandania/credit-card-fraud-detection	Credit Card Fraud Detection	2117
mjawad17/ev-charging-and-grid-optimization-data	EV Charging & Grid Optimization Data	6

```
!kaggle datasets download -d abdallahalidev/plantvillage-dataset
```

```
Dataset URL: https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset
License(s): CC-BY-NC-SA-4.0
Resuming from 2097152 bytes (2187289267 bytes left)...
100% 2.04G/2.04G [02:06<00:00, 17.3MB/s]
```

```
!unzip plantvillage-dataset.zip
```

```
inflating: plantvillage-dataset/segmented/Tomato_healthy/1253a73c-0a81-44fb-b8c0-2f5cc6c701752_05-11-2024_0540_512x512.jpg
```

```
inflating: plantvillage dataset/segmented/Tomato___healthy/1332e7ea-7a81-441b-98e9-23ee0c701735___RS_HL_0349_final
inflating: plantvillage dataset/segmented/Tomato___healthy/13688c77-f2ad-4f7c-972c-99d70652fac6___RS_HL_0458_final
inflating: plantvillage dataset/segmented/Tomato___healthy/136a69f1-2cf0-4fc6-bf40-8ed63bd3db94___RS_HL_9624_final
inflating: plantvillage dataset/segmented/Tomato___healthy/139d2f99-d25b-440d-805e-d070a399d8f1___RS_HL_0600_final
inflating: plantvillage dataset/segmented/Tomato___healthy/13a4fd9b-13eb-4fcd-af7d-b6c76fbfbf456___GH_HL_Leaf_272_f
inflating: plantvillage dataset/segmented/Tomato___healthy/13b114fe-22bf-498e-8534-4049123ef655___RS_HL_9775_final
inflating: plantvillage dataset/segmented/Tomato___healthy/13e0d497-f5fc-4fb9-9e9a-cb77bc330730___RS_HL_0397_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1421f343-ba3a-424c-ac71-b5eb9ac26c10___GH_HL_Leaf_201_f
inflating: plantvillage dataset/segmented/Tomato___healthy/14385f16-4e7f-401f-889a-8c1535ed85ba___GH_HL_Leaf_503.5
inflating: plantvillage dataset/segmented/Tomato___healthy/1457ca6a-4907-4c97-83f8-a0fa22135a27___RS_HL_0441_final
inflating: plantvillage dataset/segmented/Tomato___healthy/14768e0c-387d-40d2-96f9-031e0a45bed5___RS_HL_9667_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1479f22a-c0d6-4dac-a752-578f575f276d___GH_HL_Leaf_173.5
inflating: plantvillage dataset/segmented/Tomato___healthy/1480b935-0081-4052-a7e8-09ef0040d9b5___GH_HL_Leaf_338_f
inflating: plantvillage dataset/segmented/Tomato___healthy/148e32de-8876-45d9-92c5-43c17d7292fa___RS_HL_0189_final
inflating: plantvillage dataset/segmented/Tomato___healthy/14c040c1-d57c-43b7-9cda-74dff9e6a552___RS_HL_0241_final
inflating: plantvillage dataset/segmented/Tomato___healthy/14d46dfe-5348-428c-af6b-82b6625ce6c3___GH_HL_Leaf_483.1
inflating: plantvillage dataset/segmented/Tomato___healthy/14f9c049-aeb8-4cfb-84a7-cd39526b8b20___RS_HL_0534_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1528d8ea-c972-4da3-8f66-b4a0de6ace90___RS_HL_0573_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1531dc0c-64d5-49f2-a78a-c7b39882a47a___RS_HL_9727_final
inflating: plantvillage dataset/segmented/Tomato___healthy/15ba0299-5f38-47ed-ab19-fe6a72737705___RS_HL_9732_final
inflating: plantvillage dataset/segmented/Tomato___healthy/15cae3c8-6843-49d1-8da6-33bd014f05ce___GH_HL_Leaf_169_f
inflating: plantvillage dataset/segmented/Tomato___healthy/15daad17-78e9-45d2-9054-5a76221f6c1e___GH_HL_Leaf_246_f
inflating: plantvillage dataset/segmented/Tomato___healthy/162db449-3361-40fa-a9bf-42bb63d43781___RS_HL_9647_final
inflating: plantvillage dataset/segmented/Tomato___healthy/164d5472-31c5-4145-b30a-93198436539d___RS_HL_0103_final
inflating: plantvillage dataset/segmented/Tomato___healthy/166b90f5-3bba-4aea-9bc4-212efb57a504___RS_HL_0377_final
inflating: plantvillage dataset/segmented/Tomato___healthy/167247af-fd67-4b70-a72e-052dc5ea863b___GH_HL_Leaf_388.1
inflating: plantvillage dataset/segmented/Tomato___healthy/16927ddc-fc87-4ff2-b450-1edc2b4c6191___RS_HL_0051_final
inflating: plantvillage dataset/segmented/Tomato___healthy/16c1b712-2c55-4a8d-899f-cc0e725d4835___GH_HL_Leaf_314_f
inflating: plantvillage dataset/segmented/Tomato___healthy/16ed4e23-5218-4e8b-85df-86a7069a675d___GH_HL_Leaf_519.1
inflating: plantvillage dataset/segmented/Tomato___healthy/16f3a461-aafc-49e9-8973-39a52190985a___RS_HL_9803_final
inflating: plantvillage dataset/segmented/Tomato___healthy/16fd1407-644b-42f1-a40a-6cacd9bc0258___RS_HL_9705_final
inflating: plantvillage dataset/segmented/Tomato___healthy/170e86b9-f51d-49a6-a992-15e00c85ee5d___GH_HL_Leaf_189_f
inflating: plantvillage dataset/segmented/Tomato___healthy/1732045f-a451-4325-aa86-717a8cbdaf02___RS_HL_9698_final
inflating: plantvillage dataset/segmented/Tomato___healthy/17b48132-9a1d-45e5-9000-30ebd7248273___RS_HL_0385_final
inflating: plantvillage dataset/segmented/Tomato___healthy/17cffb66-bb7f-433c-88b4-442c1f4e8cb4___RS_HL_9708_final
inflating: plantvillage dataset/segmented/Tomato___healthy/185440e0-ca0a-4cd6-aba7-d03fae21552f___RS_HL_0141_final
inflating: plantvillage dataset/segmented/Tomato___healthy/188107b9-d61f-4084-94da-4d5f8d9eedc1___RS_HL_0553_final
inflating: plantvillage dataset/segmented/Tomato___healthy/18fa622c-8257-479a-9c81-7a6fe9a3a41b___GH_HL_Leaf_428_f
inflating: plantvillage dataset/segmented/Tomato___healthy/18fa8f7f-ef94-4a3b-9c79-5f4362323134___RS_HL_9977_final
inflating: plantvillage dataset/segmented/Tomato___healthy/190a6af7-d7b1-4265-b05b-c126a7962c93___GH_HL_Leaf_408_f
inflating: plantvillage dataset/segmented/Tomato___healthy/196e5fd8-8b4e-4a45-9db7-ae59c877e9a7___GH_HL_Leaf_284_f
inflating: plantvillage dataset/segmented/Tomato___healthy/1971e05b-fc5e-40c1-a4ff-c6a4c746693f___GH_HL_Leaf_237_f
inflating: plantvillage dataset/segmented/Tomato___healthy/19787f8b-a78e-4cf2-8fb2-3aae59b20c47___RS_HL_0456_final
inflating: plantvillage dataset/segmented/Tomato___healthy/199f4f32-6e91-4031-b861-bac1abe3452a___RS_HL_9763_final
inflating: plantvillage dataset/segmented/Tomato___healthy/19e31197-5f6d-4925-975c-884bd4a94bfe___RS_HL_0291_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1a02fa87-a194-482e-b4ac-8dec2064d793___RS_HL_0022_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1a0a5292-5e63-4afa-8611-38eef316f1e9___GH_HL_Leaf_477_f
inflating: plantvillage dataset/segmented/Tomato___healthy/1a0b9a32-3993-44e4-879f-338cd6e45a09___GH_HL_Leaf_178_f
inflating: plantvillage dataset/segmented/Tomato___healthy/1a0c4408-634e-4bd5-948d-e73ca5572976___RS_HL_0146_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1a308834-fdd7-4121-ae03-5736d8d52e26___RS_HL_9689_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1a393116-cbd1-4e3f-acfc-563e96ae1e80___RS_HL_0268_final
inflating: plantvillage dataset/segmented/Tomato___healthy/1a939012-5b9b-4dfa-aeda-ba97ec857903___GH_HL_Leaf_278_f
inflating: plantvillage dataset/segmented/Tomato___healthy/1a972145-610d-4efe-96e7-e0253c4ece4a___GH_HL_Leaf_237.1
inflating: plantvillage dataset/segmented/Tomato___healthy/1ad550a2-113e-440c-919d-85dff2f4de38___RS_HL_0331_final
```

```
!rm plantvillage-dataset.zip
```

```
!ls
```

```
'plantvillage dataset' sample_data
```

```
!ls "plantvillage dataset"
```

```
color grayscale segmented
```

```
!ls "plantvillage dataset/color" | head
```

```
Apple___Apple_scab
Apple___Black_rot
Apple___Cedar_apple_rust
Apple___healthy
Blueberry___healthy
Cherry_(including_sour)___healthy
Cherry_(including_sour)___Powdery_mildew
Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot
Corn_(maize)___Common_rust_
Corn_(maize)___healthy
```

```
import random
random.seed(0)
```

```
import numpy as np
```

```
np.random.seed(0)

import tensorflow as tf
tf.random.set_seed(0)
```

```
import os
import json
from PIL import Image

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
```

```
base_dir = "/content/plantvillage dataset/color"
```

Plant Classes

```
import os

if os.path.isdir(base_dir):
    classes = os.listdir(base_dir)
    print("Number of Classes =", len(classes))
    print(classes[:10])
else:
    print(f"Error: The directory '{base_dir}' was not found.")
    print("This often happens if the dataset was not downloaded and unzipped correctly, or if the runtime was restart")
    print("Please ensure you have run the cells to download and unzip the dataset (e.g., `!kaggle datasets download .`")
    classes = [] # Initialize classes as an empty list to avoid further errors in subsequent operations if this cell
```

```
Number of Classes = 38
['Cherry_(including_sour)___Powdery_mildew', 'Apple___Apple_scab', 'Corn_(maize)___healthy', 'Strawberry___healthy', '
```

```
# Extract plant names (before __)
plants = set([cls.split('__')[0] for cls in classes])

print("Number of Plant Types:", len(plants))
print("Plant Types:", plants)
```

```
Number of Plant Types: 14
Plant Types: {'Orange', 'Pepper, bell', 'Tomato', 'Corn (maize)', 'Squash', 'Soybean', 'Potato', 'Grape', 'Strawberry'}
```

```
img_size = 224
batch_size = 32
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

data_gen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_generator = data_gen.flow_from_directory(
    base_dir,
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='categorical',
    subset='training'
)

validation_generator = data_gen.flow_from_directory(
    base_dir,
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    shuffle=False
)
```

)

Found 43456 images belonging to 38 classes.
Found 10849 images belonging to 38 classes.

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(img_size, img_size, 3)),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dense(train_generator.num_classes, activation='softmax')
])
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `activity\_regularizer` argument to the Conv2D layer. The `activity\_regularizer` argument will be ignored. (**kwargs)

model.summary()

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
flatten (Flatten)	(None, 186624)	0
dense (Dense)	(None, 256)	47,776,000
dense_1 (Dense)	(None, 38)	9,766

Total params: 47,805,158 (182.36 MB)
Trainable params: 47,805,158 (182.36 MB)
Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // batch_size,
    epochs=5,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // batch_size
)
```

Epoch 1/5
1358/1358 ————— 96s 66ms/step - accuracy: 0.7435 - loss: 0.9078 - val_accuracy: 0.8697 - val_loss: 0.41
Epoch 2/5
1358/1358 ————— 97s 72ms/step - accuracy: 0.9185 - loss: 0.2526 - val_accuracy: 0.8699 - val_loss: 0.40
Epoch 3/5
1358/1358 ————— 91s 67ms/step - accuracy: 0.9628 - loss: 0.1130 - val_accuracy: 0.8833 - val_loss: 0.42
Epoch 4/5
1358/1358 ————— 89s 66ms/step - accuracy: 0.9753 - loss: 0.0743 - val_accuracy: 0.8955 - val_loss: 0.44
Epoch 5/5
1358/1358 ————— 90s 66ms/step - accuracy: 0.9806 - loss: 0.0598 - val_accuracy: 0.8834 - val_loss: 0.52

Model Evaluation

```
# Model Evaluation
print("Evaluating model...")
val_loss, val_accuracy = model.evaluate(
    validation_generator,
    steps=validation_generator.samples // batch_size
)

print(f"Validation Accuracy: {val_accuracy * 100:.2f}%")
```

```
Evaluating model...
339/339 ————— 17s 50ms/step - accuracy: 0.8834 - loss: 0.5210
Validation Accuracy: 88.34%
```

Plot Accuracy & Loss Curves

```
# Plot training & validation accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()

# Plot training & validation loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

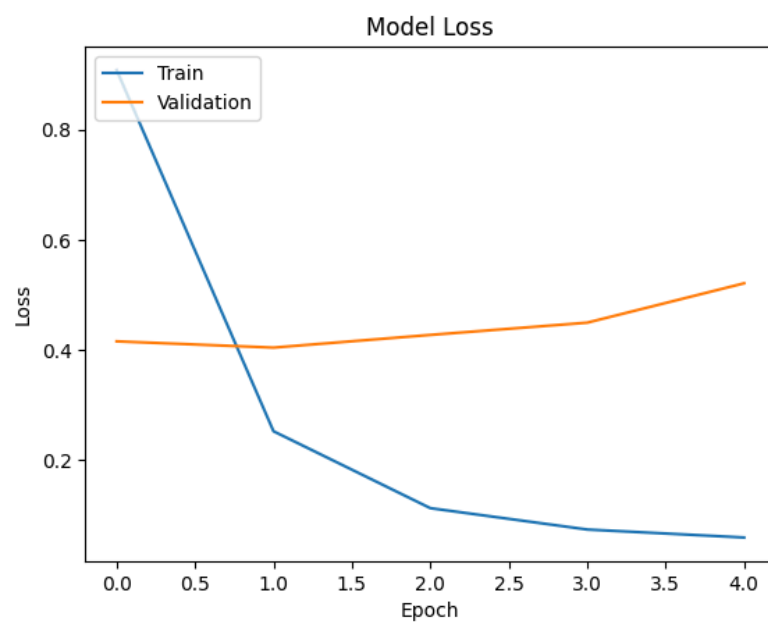
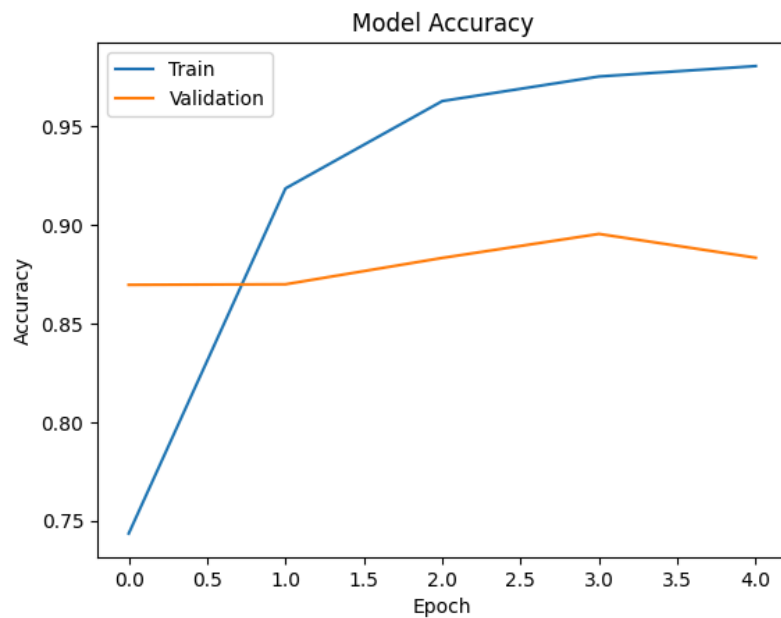


Image Preprocessing Function

```
from PIL import Image
import numpy as np

def load_and_preprocess_image(image_path, target_size=(224, 224)):
    img = Image.open(image_path)
    img = img.resize(target_size)
    img_array = np.array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = img_array.astype('float32') / 255.0
    return img_array
```

Prediction Function

```
def predict_image_class(model, image_path, class_indices):
    preprocessed_img = load_and_preprocess_image(image_path)
    predictions = model.predict(preprocessed_img)
    predicted_class_index = np.argmax(predictions, axis=1)[0]
    predicted_class_name = class_indices[predicted_class_index]
    return predicted_class_name
```

Create Class Index Mapping

```
# Create mapping from index to class name
class_indices = {v: k for k, v in train_generator.class_indices.items()}
```

Save Class Indices

```
import json
json.dump(class_indices, open('class_indices.json', 'w'))
```

Test the Predictive System

```
import os

# Define the directory for Potato__Late_blight within the dataset
potato_lateblight_dir = os.path.join(base_dir, 'Potato__Late_blight')

# Get a list of all files in that directory
image_files = os.listdir(potato_lateblight_dir)

# Select the first image file for testing (ensure there's at least one file)
if image_files:
    image_path = os.path.join(potato_lateblight_dir, image_files[0])
else:
    print(f"No images found in {potato_lateblight_dir}")
    image_path = None # Set to None or handle appropriately

if image_path:
    predicted_class_name = predict_image_class(
        model,
        image_path,
        class_indices
    )

    print("Predicted Class Name:", predicted_class_name)
else:
    print("Could not perform prediction as no image path was found.")

1/1  1s 1s/step
Predicted Class Name: Potato__Late_blight
```

Save Model Locally

```
model.save('plant_disease_prediction_model.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This fi

```
model.save("plant_disease_prediction_model.keras")
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
!mkdir -p /content/drive/MyDrive/PlantDiseaseModel
```

```
!cp plant_disease_prediction_model.keras /content/drive/MyDrive/PlantDiseaseModel/
!cp class_indices.json /content/drive/MyDrive/PlantDiseaseModel/
```

```
!ls /content/drive/MyDrive/PlantDiseaseModel
```

```
class_indices.json  plant_disease_prediction_model.keras
```

```

y_true = validation_generator.classes

y_pred_probs = model.predict(validation_generator, verbose=1)
y_pred = np.argmax(y_pred_probs, axis=1)

```

340/340 ————— 21s 59ms/step

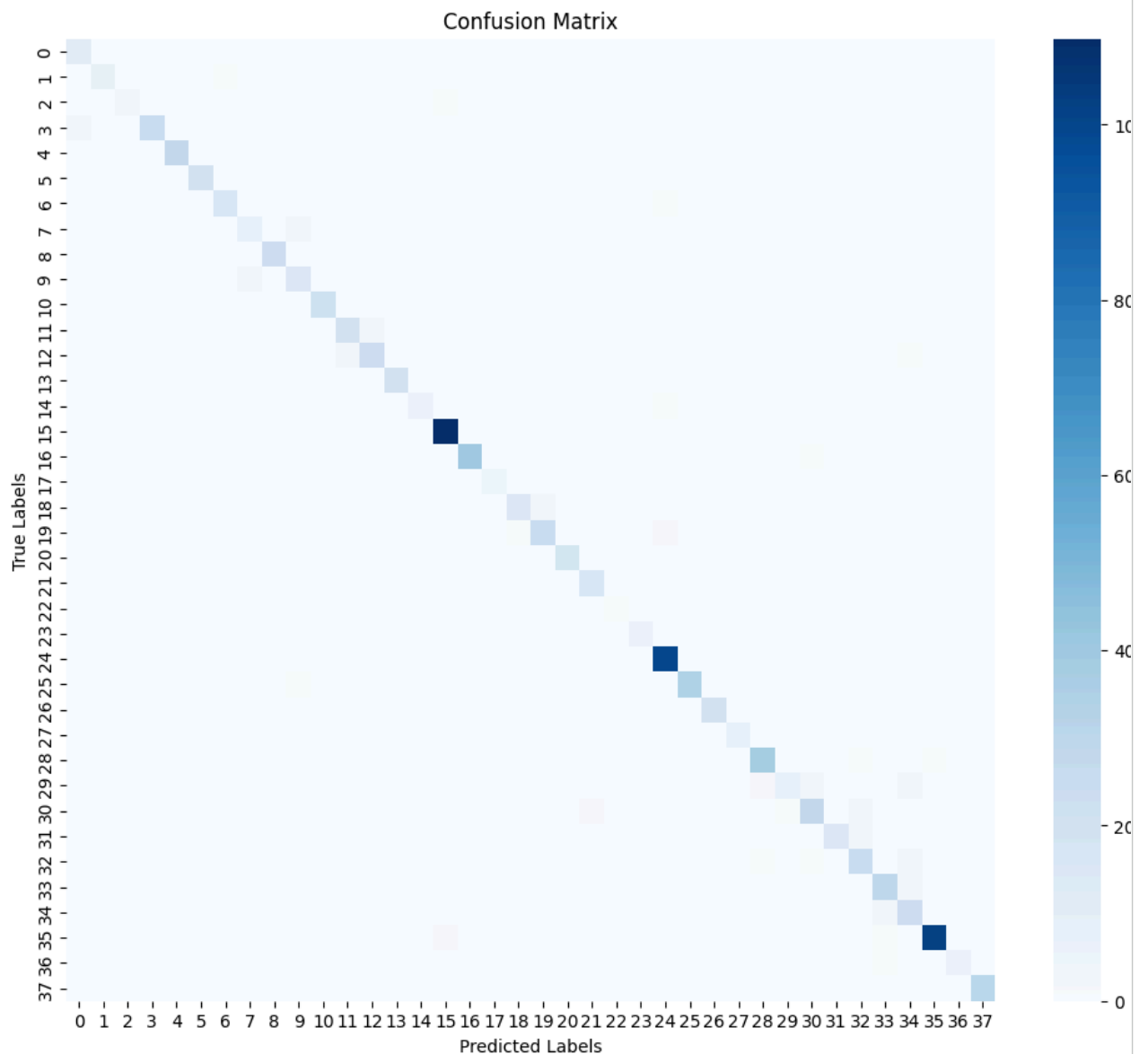
```

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(12,10))
sns.heatmap(cm, cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.show()

```



```

import pandas as pd
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')

metrics_df = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],
    "Value": [
        round(0.8785, 4),
        round(precision, 4),

```



```

        round(recall, 4),
        round(f1, 4)
    ]
})

print(metrics_df)

```

	Metric	Value
0	Accuracy	0.8785
1	Precision	0.8884
2	Recall	0.8834
3	F1-Score	0.8831

```

from tensorflow.keras.metrics import TopKCategoricalAccuracy
import tensorflow as tf

top5 = TopKCategoricalAccuracy(k=5)

# Convert y_true to one-hot encoded format
num_classes = train_generator.num_classes
y_true_one_hot = tf.one_hot(y_true, depth=num_classes)

top5.update_state(y_true_one_hot, y_pred_probs)

print("Top-5 Accuracy:", top5.result().numpy())

```

Top-5 Accuracy: 0.99133563

```

import random
from tensorflow.keras.preprocessing.image import load_img, img_to_array

# Get class labels mapping
class_labels = list(train_generator.class_indices.keys())

# Function to display predictions
def show_predictions(generator, model, num_images=5):
    plt.figure(figsize=(15,5))

    for i in range(num_images):
        idx = random.randint(0, len(generator.filepaths)-1)
        img_path = generator.filepaths[idx]

        img = load_img(img_path, target_size=(224,224))
        img_array = img_to_array(img)/255.0
        img_array = np.expand_dims(img_array, axis=0)

        pred = model.predict(img_array)
        pred_class = class_labels[np.argmax(pred)]
        true_class = img_path.split('/')[-2]

        plt.subplot(1, num_images, i+1)
        plt.imshow(img)
        plt.title(f"Pred: {pred_class}\nActual: {true_class}")
        plt.axis('off')

    plt.show()

show_predictions(validation_generator, model)

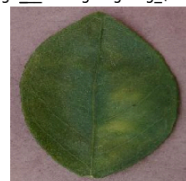
```

```

1/1 ————— 0s 39ms/step
1/1 ————— 0s 42ms/step
1/1 ————— 0s 40ms/step
1/1 ————— 0s 49ms/step
1/1 ————— 0s 40ms/step

```

Pred: Orange__Haunglongbing_(Citrus_greening) Pred: Citrus_greening Pred: Citrus_greening Pred: Citrus_greening Pred: Citrus_greening
 Actual: Orange__Haunglongbing_(Citrus_greening) Actual: Citrus_greening Actual: Citrus_greening Actual: Citrus_greening Actual: Citrus_greening



```

import os
import matplotlib.pyplot as plt
from PIL import Image

```

```
base_dir = "/content/plantvillage dataset/color"

classes = os.listdir(base_dir)

plt.figure(figsize=(15,10))

for i, cls in enumerate(classes[:9]):
    img_path = os.path.join(base_dir, cls, os.listdir(os.path.join(base_dir, cls))[0])
    img = Image.open(img_path)

    plt.subplot(3,3,i+1)
    plt.imshow(img)
    plt.title(cls)
    plt.axis('off')

plt.show()
```

Cherry_(including_sour)___Powdery_mildew



Apple___Apple_scab



Corn_(maize)___healthy

